



PROG6212 ST10490858

[Document subtitle]

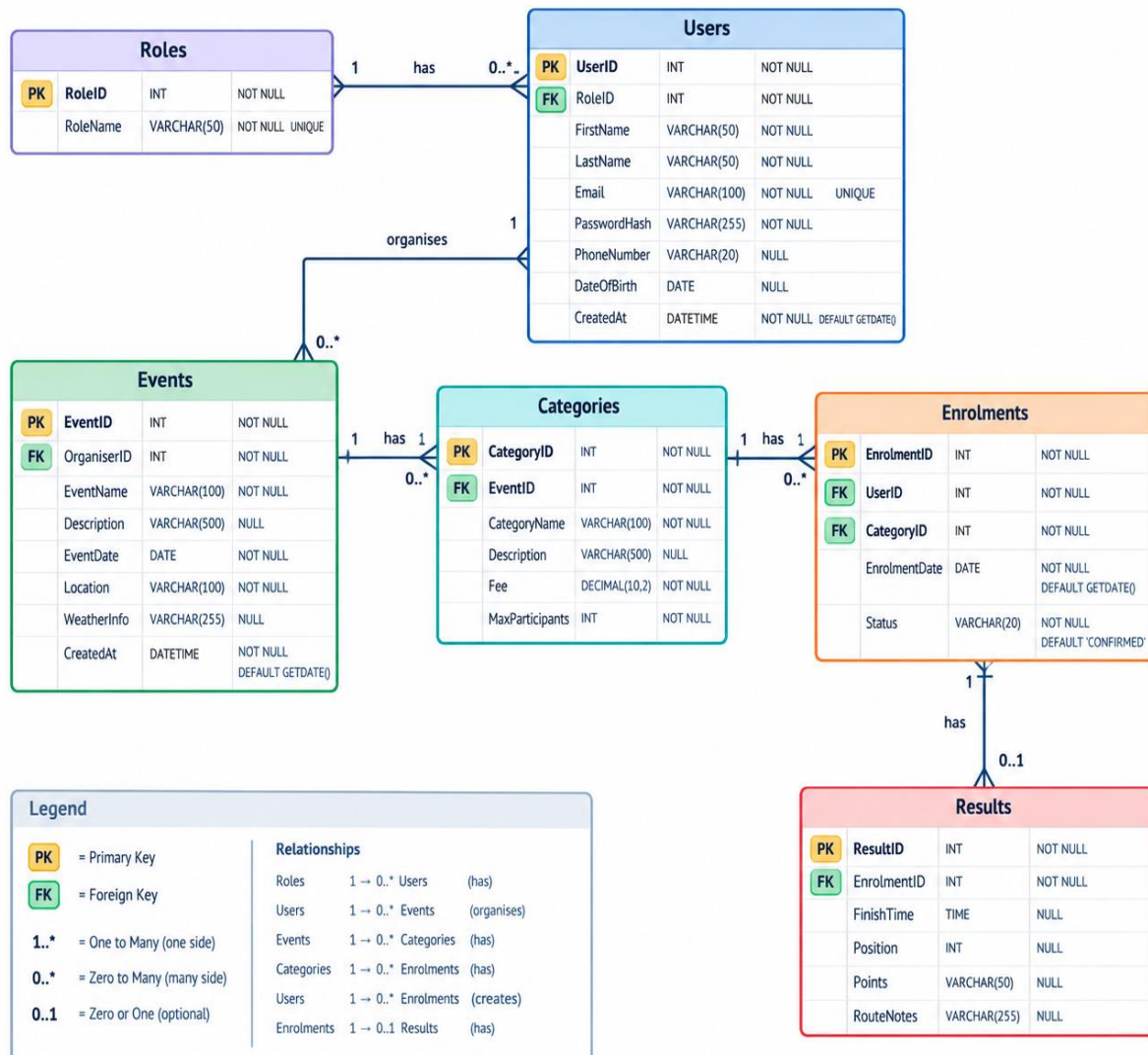


SEPTEMBER 1, 2026

ROSEBANK COLLEGE
PTA CAMPUS

SECTION A

RaceDay – Entity Relationship Diagram (ERD)



Relationship Summary

- Roles 1 → 0..* Users: each user is assigned a role.
- Users 1 → 0..* Events: an organiser can create and manage multiple events.
- Events 1 → 0..* Categories: each event can contain multiple race categories.
- Categories 1 → 0..* Enrolments: participants enrol in a selected category.
- Users 1 → 0..* Enrolments: a participant can have multiple event enrolments.
- Enrolments 1 → 0..1 Results: an enrolment may have one result after the event.

SECTION B – API ENDPOINT PLAN

This endpoint plan defines the RESTful API to be implemented in Part 2. It covers authentication, user profiles, events, categories, enrolments and results and applies the required Organiser and Participant roles.

Roles

Public – no authentication required.

Any logged-in user – valid authentication token required.

Participant – authenticated user with the Participant role.

Organiser – authenticated user with the Organiser role.

API Endpoint Table

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/auth/register	Creates a new user account.	Public	{ firstName, lastName, email, password, roleId }	201 Created; 400 Bad Request; 409 Conflict if email exists.
POST	/api/auth/login	Authenticates a user and returns an access token and role.	Public	{ email, password }	200 OK with token; 401 Unauthorized for invalid credentials.
GET	/api/users/me	Returns the authenticated user's profile.	Any logged in	None	200 OK; 401 Unauthorized; 404 Not Found.
PUT	/api/users/me	Updates the authenticated user's profile.	Any logged in	{ firstName, lastName, email, phoneNumber, dateOfBirth }	200 OK; 400 Bad Request; 401 Unauthorized; 409 Conflict.
GET	/api/events	Lists upcoming events	Public	None	200 OK with event list; 500 Internal

		available for browsing.			Server Error.
GET	/api/events/{eventId}	Returns details of one event.	Public	None	200 OK; 404 Not Found.
POST	/api/events	Creates a new RaceDay event.	Organiser	{ eventName, description, eventDate, location, weatherInfo, routeInfo }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden.
PUT	/api/events/{eventId}	Updates an event managed by the organiser.	Organiser	{ eventName, description, eventDate, location, weatherInfo, routeInfo }	200 OK; 400 Bad Request; 403 Forbidden; 404 Not Found.
DELETE	/api/events/{eventId}	Deletes an event managed by the organiser.	Organiser	None	204 No Content; 403 Forbidden; 404 Not Found; 409 Conflict.
GET	/api/events/{eventId}/categories	Lists all categories for an event.	Public	None	200 OK with category list; 404 Not Found.
POST	/api/events/{eventId}/categories	Creates a category for an event.	Organiser	{ categoryName, description, entryFee }	201 Created; 400 Bad Request; 403 Forbidden; 404 Not Found.
GET	/api/categories/{categoryId}	Returns details of a category.	Public	None	200 OK; 404 Not Found.

PUT	/api/categories/{categoryId}	Updates an event category.	Organiser	{ categoryName, description, entryFee }	200 OK; 400 Bad Request; 403 Forbidden; 404 Not Found.
DELETE	/api/categories/{categoryId}	Deletes a category when no enrolment conflict exists.	Organiser	None	204 No Content; 403 Forbidden; 404 Not Found; 409 Conflict.
POST	/api/enrolments	Enrols the authenticated participant in a category.	Participant	{ categoryId }	201 Created; 400 Bad Request; 403 Forbidden; 404 Not Found; 409 Conflict.
GET	/api/enrolments/me	Returns the participant's own event enrolments.	Participant	None	200 OK with enrolments; 401 Unauthorized.
GET	/api/events/{eventId}/enrolments	Returns all enrolments for an event.	Organiser	None	200 OK with enrolments; 403 Forbidden; 404 Not Found.
GET	/api/enrolments/{enrolmentId}	Returns one enrolment for an authorised user.	Any logged in	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found.
PUT	/api/enrolments/{enrolmentId}/status	Updates an enrolment status such	Organiser	{ status }	200 OK; 400 Bad Request;

		as Confirmed or Cancelled.			403 Forbidden; 404 Not Found.
DELETE	/api/enrolments/{enrolmentId}	Cancels a participant's own enrolment.	Participant	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found.
POST	/api/results	Records a participant's result for a completed enrolment.	Organiser	{ enrolmentId, finishTime, position, resultStatus }	201 Created; 400 Bad Request; 403 Forbidden; 404 Not Found; 409 Conflict.
GET	/api/results/me	Returns the participant's personal results history.	Participant	None	200 OK with results; 401 Unauthorized.
GET	/api/events/{eventId}/results	Returns all results recorded for an event.	Organiser	None	200 OK with results; 403 Forbidden; 404 Not Found.
GET	/api/results/{resultId}	Returns one race result.	Any logged in	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found.
PUT	/api/results/{resultId}	Corrects or updates a race result.	Organiser	{ finishTime, position, resultStatus }	200 OK; 400 Bad Request; 403 Forbidden; 404 Not Found.

DELETE	/api/results/{resultId}	Deletes an incorrectly captured result.	Organiser	None	204 No Content; 403 Forbidden; 404 Not Found.
--------	-------------------------	---	-----------	------	---

SECTION C: SQL DATABASE IMPLEMENTATION

SQL Script Location

The RaceDay SQL database script was created using SQL Server Management Studio (SSMS). The script creates the `RaceDayDB` database and includes all six tables used in the system: `Roles`, `Users`, `Events`, `Categories`, `Enrolments`, and `Results`.

The SQL script will be saved as:

`/docs/RaceDayDB.sql`

The script first checks whether the `RaceDayDB` database already exists. If it does, the database is removed and recreated. This allows the script to be tested on a clean SQL Server database. The POE requires the SQL script to be saved as a `.sql` file and committed inside the `/docs` folder.

Sample Data

The database is populated with realistic sample data using `INSERT` statements.

The `Roles` table contains two roles:

- * Organiser
- * Participant

The `Users` table contains two Organisers and two Participants. The Organisers are Thabo Mokoena and Lerato Naidoo, while the Participants are Sipho Dlamini and Anele Jacobs.

Three events are included in the `Events` table:

- * Johannesburg Spring Run
- * Cape Town Coastal Cycle
- * Durban Charity Walk

Each event has categories. The categories include 10 km Run, 21 km Run, 40 km Cycle, 80 km Cycle, 5 km Walk, and 10 km Walk. Entry fees are also included for the categories.

Sample enrolments are included to show how Participants are linked to different event categories. Two sample results are also included, containing finish times, positions and result statuses.

Key Design Decisions

1. Identity primary keys

`IDENTITY(1,1)` is used for the primary keys in all six tables. This automatically generates unique ID values when new records are inserted.

2. Primary and foreign keys

Primary keys are used to uniquely identify records. Foreign keys are used to create relationships between the tables. For example, `Users.RoleID` links users to the `Roles` table, while `Events.OrganiserID` links an event to its organiser.

3. NOT NULL constraints

Important fields are set to `NOT NULL`. This prevents required information such as names, email addresses, event dates and locations from being left empty.

4. UNIQUE constraints

The `Email` field in the `Users` table is unique so that two users cannot register using the same email address. The `RoleName` field is also unique. The `Enrolments` table uses a composite unique constraint on `UserID` and `CategoryID` to prevent a participant from enrolling in the same category more than once.

5. DEFAULT values

Default values are used for fields such as `CreatedAt`, `EnrolmentDate`, `EntryFee`, and `Status`. This reduces the amount of information that needs to be entered manually when creating records.

6. CHECK constraints

CHECK constraints are used to control valid data. For example, `EntryFee` cannot be negative, enrolment status can only be `Pending`, `Confirmed`, or `Cancelled`, and race positions must be greater than zero.

7. Relationships between entities

The database uses foreign keys to represent the relationships between Roles, Users, Events, Categories, Enrolments and Results. This keeps the data organised and reduces unnecessary duplication.

8. Sample data for testing Sample records were added so that the database can be tested using realistic RaceDay information. The final part of the script also uses `SELECT` statements and record counts to check that the data was inserted correctly.

The database design follows the Section C requirements that the SQL script must contain the entities from the ERD, include the required constraints, contain realistic seed data, and run successfully on a clean SQL Server instance.

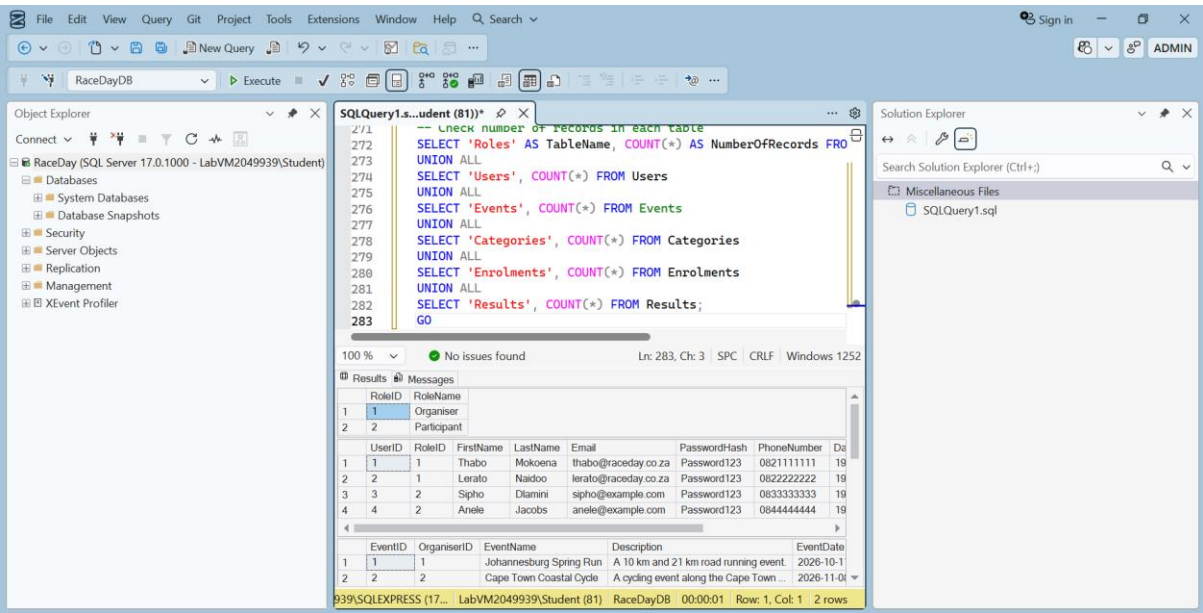


Figure 1Screenshot1

	TableName	NumberOfRecords
1	Roles	2
2	Users	4
3	Events	3
4	Categories	6
5	Enrolments	4
6	Results	2

Figure 2Screenshot2

